

18. Un tableau : plusieurs cases sous un seul nom

Cinq scores, ce n'est pas cinq variables : c'est un tableau de cinq cases, et une boucle.

1. Ce qu'il faut comprendre

`uint8_t scores[5]` réserve **cinq cases qui se suivent** en mémoire, sous un seul nom. On en désigne une par son **indice**, entre crochets : `scores[0]`, `scores[1]`... jusqu'à `scores[4]`.

Le premier indice est 0. Un tableau de 5 cases va de 0 à 4 ; `scores[5]` n'existe pas — et la console ne t'arrêtera pas : elle lira ou écrira la case d'après, qui appartient à une autre variable. C'est pour cela que la boucle s'écrit `i < 5`, et jamais `i <= 5`.

L'indice peut être **calculé** : `scores[i]`. C'est tout l'intérêt. Avec cinq variables `score0`... `score4`, il faudrait cinq lignes d'affichage ; avec un tableau, **une boucle** — et elle marcherait pareil pour 50 joueurs.

`= { 12, 40, 7, 99, 25 }` donne les valeurs de départ, dans l'ordre des indices. Plus tard, `scores[2] = 50;` change une seule case, sans toucher aux autres.

`sizeof(scores)` donne la taille du tableau en octets : 5. On l'utilise pour ne pas écrire le 5 deux fois.

À toi : ajoute un sixième joueur. Grâce à `sizeof`, la boucle n'a pas besoin de changer.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **660 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      66 o ← la console, à l'allumage — la mise en route de la console
main()         136 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← ecran() — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase    10 o ← texte(), nombre() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         240 o ← CopierTuiles — 15 lettres sur 43
"JOUEUR"       6 o ← texte() — un texte gravé
scores_depart_0 5 o ← démarrage — une table gravée
```

Lettres gravées : EJORU0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 2    60 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
texte() × 1     25 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
```

```
ecran()  × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
image()  × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : range 5 scores dans UN tableau, en change un, et les affiche tous.
//
// Ce qui est nouveau : le TABLEAU. Plusieurs cases sous un seul nom.
//  uint8_t scores[5]  = 5 cases uint8_t, nommées scores[0], scores[1] ... scores[4].
//  Le numéro entre crochets s'appelle l'INDICE. Il commence à 0, pas à 1 !

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. UN TABLEAU — SCORES

```
uint8_t scores[5] = { 12, 40, 7, 99, 25 }; // Les 5 valeurs de départ, dans l'ordre :
// scores[0] = 12, scores[1] = 40,
// scores[2] = 7,  scores[3] = 99, scores[4] = 25.
// Écrit HORS de main : le tableau existe
// pendant tout le jeu.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    ecran(0);                // Dessin écran éteint.
    scores[2] = 50;          // la troisième case : l'indice 2. Elle passe de 7 à 50.

    for (uint8_t i = 0; i < sizeof(scores); i++) { // sizeof(scores) = la taille du tableau : 5.
                                                // Donc i = 0, 1, 2, 3, 4 : une fois par case.
        texte(2, 3 + i * 2, "JOUEUR");          // La ligne : 3 + i * 2 → 3, 5, 7, 9, 11
                                                // (une ligne vide entre deux joueurs).
        nombre(9, 3 + i * 2, i, 1);              // Le numéro du joueur : i.
        nombre(13, 3 + i * 2, scores[i]);        // scores[i] : la case numéro i.
                                                // i = 0 → 12 ; i = 2 → 50 ; i = 4 → 25.
    }

    ecran(1);                // On rallume : tout apparaît.

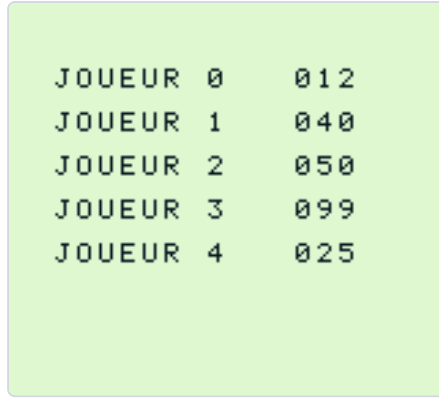
    while (true) {           // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Cinq lignes JOUEUR 0 à JOUEUR 4, avec 012, 040, 050, 099, 025.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les cinq cases sont affichées dans l'ordre (012 040 050 099 025)
- ✓ le premier joueur porte l'indice 0
- ✓ le dernier, l'indice 4

← [17. Une boucle qui dure plusieurs images](#)

[sommaire](#)

[19. Parcourir : le total, le plus grand, le plus petit](#) →